

ReadingsRepositories.mjs

Now we're at the lowest layer of the main Lambda.

This file answers:

How do I actually communicate with AWS?

The service shouldn't contain DynamoDB syntax such as:

```
1 KeyConditionExpression
2 ExpressionAttributeValues
3 UpdateExpression
```

That's repository work.

AWS imports

At the top you'll have imports like:

```
1 import {
2   DynamoDBClient
3 } from "@aws-sdk/client-dynamodb";
```

and:

```
1 import {
2   DynamoDBDocumentClient,
3   GetCommand,
4   PutCommand,
5   QueryCommand,
6   UpdateCommand,
7   DeleteCommand
8 } from "@aws-sdk/lib-dynamodb";
```

Each command corresponds roughly to an operation:

```
1 GetCommand
2 → retrieve one
3
4 PutCommand
5 → create/save
6
7 QueryCommand
8 → search/query multiple
9
10 UpdateCommand
11 → change existing
12
13 DeleteCommand
14 → delete
```

You also import:

```
1 SNSClient
```

because this repository publishes excursion events.

Environment variables

You'll have values such as:

```
1 const TABLE_NAME =
2   process.env.TABLE_NAME;
```

and:

```
1 const EXCURSION_TOPIC_ARN =
2   process.env.EXCURSION_TOPIC_ARN;
```

Where do these come from?

Terraform.

Your `lambda.tf` contains environment variables similar to:

```
1 environment {
2   variables = {
3     TABLE_NAME =
4       aws_dynamodb_table.readings_by_id.name
5
6     EXCURSION_TOPIC_ARN =
7       aws_sns_topic.excursions.arn
8
9     BRANCH_RECORDED_AT_INDEX =
10      var.readings_gsi_name
11   }
12 }
```

That's an important Terraform ↔ JavaScript connection:

```
1 Terraform
2 lambda.tf
3   ↓
4 sets TABLE_NAME
5   ↓
6 AWS Lambda environment
7   ↓
8 JavaScript
9 process.env.TABLE_NAME
```

So you don't hard-code:

```
1 umar-coldchain-readings-terraform
```

inside JavaScript.

DynamoDB client

This:

```
1 const client =  
2   new DynamoDBClient({  
3     region: REGION  
4   });
```

creates the low-level AWS connection.

Then:

```
1 const dynamo =  
2   DynamoDBDocumentClient.from(  
3     client  
4   );
```

gives you the easier document-style API where normal JavaScript objects can be used.

findReadingById()

This uses:

```
1 new GetCommand({  
2   TableName: TABLE_NAME,  
3  
4   Key: {  
5     readingId  
6   }  
7 })
```

Why `GetCommand` ?

Because:

```
1 readingId = DynamoDB primary key
```

So DynamoDB can directly find:

```
1 readingId ABC
```

without scanning the whole table.

This is one of the biggest improvements from your old design.

queryBranchReadings()

GET-all can't use `readingId` , because we want:

```
1 all records belonging to BRANCH-001
```

So this uses your GSI:

```
1 branchId + recordedAt
```

The repository creates:

```
1 new QueryCommand(...)
```

with:

```
1 IndexName =  
2 branch-recordedAt-index
```

and:

```
1 branchId = BRANCH-001
```

Optional `from / to` conditions are added to `recordedAt`.

And:

```
1 ScanIndexForward: false
```

means:

Return sort-key results in descending order.

Because the sort key is:

```
1 recordedAt
```

that gives:

```
1 newest  
2 ↓  
3 older  
4 ↓  
5 oldest
```

saveReading()

This uses:

```
1 new PutCommand(...)
```

to create the item.

It also has:

```
1 ConditionExpression:  
2 "attribute_not_exists(readingId)"
```

which tells DynamoDB:

Only perform this Put if this primary key isn't already present.

That's your duplicate protection.

updateReading()

This uses:

```
1 new UpdateCommand(...)
```

and updates:

```
1 minTempC
2 maxTempC
3 recordedBy
4 status
```

using:

```
1 UpdateExpression
```

It also returns:

```
1 ReturnValues:
2   "ALL_NEW"
```

meaning:

Give me the complete item after the update.

That's why PATCH can return the updated reading to Postman.

deleteReadingFromDatabase()

This uses:

```
1 new DeleteCommand({
2   TableName: TABLE_NAME,
3
4   Key: {
5     readingId
6   }
7 })
```

So:

```
1 Service
2 deleteReading()
3   ↓
4 Repository
5 deleteReadingFromDatabase()
6   ↓
7 DeleteCommand
8   ↓
9 DynamoDB
```

publishExcursion()

This uses:

```
1 new PublishCommand(...)
```

with:

```
1 TopicArn = EXCURSION_TOPIC_ARN
```

and turns the JavaScript object into JSON:

```
1 JSON.stringify(message)
```

Then:

```
1 Repository
2   ↓
3 SNS PublishCommand
4   ↓
5 SNS topic
```

The main Lambda's work can now lead into the second Lambda.